

Game of Life: Implementação com OpenMP

José Alan Teixeira¹

¹Programa de Pós-graduação em Ciências da Computação

Universidade estadual de Feira de Santana (UEFS)

Feira de Santana – BA – Brasil

{joseasteixeira7}@gmail.com

1. Introdução

O game of life é um autônomo celular criado pelo britânico John Conway, consiste em um jogo que funciona sem a interação com o usuário, possui o formato de uma matriz na qual cada célula pode estar em dois estados, vivas ou mortas. Para que o jogo possa ser executado de forma correta é necessário que seja definido um estado inicial especificando quais células estão vivas e quais estão mortas no início do jogo, então a partir do estado inicial são aplicadas as seguintes regras:

- As células vivas que possuem menos de dois vizinhos vivos morrem por subpopulação;
- As células vivas que possuem exatamente dois vizinhos ou três, vivem para a próxima geração;
- As células vivas que possuem quatro vizinhos ou mais morrem por superpopulação;
- As células mortas com exatamente três vizinhos vivos, vivem para próxima geração.

2. Arquitetura Geral

Neste trabalho o Jogo da Vida de Conway foi adaptado para receber configurações iniciais por meio de arquivo de texto, e ter como saída imagens em formato portable bitmap (PBM), e medição do tempo de execução. A implementação é organizada em múltiplos arquivos para organização do código:

- main.c: ponto de entrada do programa;
- grid.c: gerenciamento da grade (alocação, inicialização e liberação);
- logic.c: lógica de evolução das gerações;
- image.c: exportação da grade em formato de imagem PBM;
- gol.h: cabeçalho compartilhado com declarações e estruturas.

3. Implementação Serial

A seguir é detalhada a função de cada arquivo que compõe a implementação do jogo da vida.

3.1. main.c - Execução principal

Principais responsabilidades:

- Ler configurações do arquivo .txt:

WIDTH, HEIGHT, ITERACOES, INTERVALO

- Inicializar e alocar a grade
- Executar o loop de gerações
- Salvar imagens periodicamente
- Medir e registrar o tempo de execução

Funções:

ler_configuracoes: Lê os parâmetros iniciais do arquivo de texto (largura, altura, número de iterações e intervalo de salvamento das imagens).

executar_simulacao: Executa o loop principal da simulação e chama `gen_next()` a cada geração. A cada intervalo gera e salva uma imagem PBM.

salvar_tempo_em_arquivo: Calcula o tempo total e salva no arquivo `tempos_serial.txt`.

3.2. grid.c - Gerenciamento da grade

Trata da memória e da inicialização do universo de células.

Funções:

allocate_grid: Aloca dinamicamente uma matriz de ponteiros para células (`Cell** grid`), com base nos valores globais `WIDTH` e `HEIGHT`.

load_grid_from_file: Ler o estado inicial da grade de um arquivo em texto.

Caso não seja encontrado o arquivo de texto com o estado inicial a função `init_grid` é chamada.

init_grid: Inicializa a grade com células vivas de forma aleatória, com baixa densidade.

free_grid: Libera corretamente a memória alocada pela grade.

3.3. logic.c - Regras do Jogo da Vida

Define a lógica de transição de uma geração para a próxima.

Funções:

3.4. gen_next: Responsável por aplicar as regras do Jogo da Vida

- Qualquer célula viva com menos de 2 ou mais de 3 vizinhos vivos morre.
- Qualquer célula viva com 2 ou 3 vizinhos sobrevive.
- Qualquer célula morta com exatamente 3 vizinhos vivos nasce.

A função utiliza uma grade auxiliar `new_grid` para armazenar a nova geração antes de substituir a original.

Observação: As bordas da grade são consideradas periódicas (efeito toroidal), simulando um espaço contínuo.

3.5. image.c - Geração de imagens PBM

Permite visualizar o estado da grade a cada intervalo de iteração.

save_pbm: Gera um arquivo .pbm chamado gol_X.pbm, onde X é o número da geração. A imagem utiliza o formato PBM (preto e branco), sendo:

- 1 para células vivas
- 0 para células mortas

3.5. gol.h - Cabeçalho da aplicação

Define as estruturas e declarações comuns usadas pelos módulos.

Estruturas: State e Cell;

Declarações globais: **grid; extern int WIDTH, HEIGHT;

Funções declaradas:

- allocate_grid, free_grid, init_grid
- gen_next
- save_pbm

4. Implementação Paralela

A versão paralela foi desenvolvida a partir do código da versão serial através da utilização do OpenMP, as alterações foram realizadas em três pontos do código das quais duas são no arquivo logic.c, o código foi paralelizado inicialmente no momento de criação da grade temporária:

```
#pragma omp parallel for
for (int i = 0; i < HEIGHT; i++) {
    new_grid[i] = malloc(WIDTH * sizeof(Cell));
}
```

Este trecho aloca memória para cada linha da grade temporária new_grid em paralelo. Cada iteração do loop (i) é independente das outras, então múltiplas threads podem alocar suas respectivas linhas simultaneamente. Isso acelera a inicialização da estrutura de dados, principalmente em grades grandes, ao distribuir o trabalho de alocação entre os núcleos da CPU.

No segundo ponto de paralelização como a grade é dividida em quatro grupos de colunas cada um desses grupos são distribuídos de forma estática entre as threads, quando existir mais de uma thread. A forma de escalonamento estático foi escolhida porque os grupos de colunas possuem aproximadamente o mesmo tamanho consequentemente o tempo de processamento será parecido para cada bloco, não havendo um grande desequilíbrio no tempo de execução.

```

// Paraleliza por blocos de colunas
#pragma omp parallel for schedule(static)
for (int b = 0; b < granularidade; b++) {
    // Define início e fim do bloco considerando o resto
    int start_col = b * base_block_width + (b < remainder ? b : remainder);
    int block_width = base_block_width + (b < remainder ? 1 : 0);
    int end_col = start_col + block_width;

    // Aplica as regras do jogo apenas nas colunas do bloco
    for (int i = 0; i < HEIGHT; i++) {
        for (int j = start_col; j < end_col; j++) {
            int alive_count = 0;
            for (int k = -1; k <= 1; k++) {
                for (int l = -1; l <= 1; l++) {
                    if (k == 0 && l == 0) continue;

                    int row = (i + k + HEIGHT) % HEIGHT;
                    int col = (j + l + WIDTH) % WIDTH;

                    if (grid[row][col].state == ALIVE) {
                        alive_count++;
                    }
                }
            }

            // Aplica as regras de transição
            if (grid[i][j].state == ALIVE) {
                new_grid[i][j].state = (alive_count < 2
                    || alive_count > 3) ? DEAD : ALIVE;
            } else {
                new_grid[i][j].state = (alive_count == 3) ? ALIVE : DEAD;
            }
        }
    }
}

```

Este é o ponto mais importante de paralelização pois aplica as regras de evolução das células para serem processadas em blocos de colunas divididos entre as threads. Cada thread recebe um bloco de colunas e atualiza o estado de todas as células daquele bloco. No momento da divisão dos blocos de colunas caso a quantidade de colunas não seja divisível pelo número predefinido de blocos as colunas que sobram na são alocadas de forma que a quantidade de colunas por blocos fique o mais parecidos possível.

Quando a atualização do estado do jogo da vida, o acesso à grade original é somente de leitura, então não há concorrência nesse acesso. A escrita ocorre apenas na `new_grid`, e cada thread escreve em regiões distintas (sem conflito), então não há necessidade de sincronização manual. Para as células das bordas de cada bloco de colunas assim como nas bordas do topo e final da grade como possui o efeito de bordas infinitas, é necessário comunicação com as células vizinhas para obter o estado dos vizinhos para atualizar a célula.

O código também foi paralelizado no arquivo `grid.c` quando a grade é alocada dinamicamente com base nas dimensões informadas.

```
#pragma omp parallel for
for (int i = 0; i < HEIGHT; i++) {
    grid[i] = malloc(WIDTH * sizeof(Cell));
}
```

Cada thread aloca linhas diferentes da grade, como cada iteração é independente, o trabalho é facilmente dividido. Há um ganho de desempenho que varia com o tamanho da grade e do número de threads.

A arquitetura utilizada na paralelização do jogo da vida foi a Phase Parallel Paradigm, na qual o computo de cada pacote de colunas acontece independentemente, e ao fim todos são sincretizados antes de iniciar outro computo.

A granularidade utilizada permanece a mesma do código serial sendo a grade dividida em quatro blocos de colunas. Caso a granularidade seja aumentada o desempenho será impactado pois aumentará a comunicação, e caso a granularidade seja diminuída o desempenho também será reduzido pois haverá subutilização dos recursos computacionais.

5. Arquivos auxiliares

5.1. Arquivos de entrada

- Config.txt - fornece parâmetros de dimensões da grade, quantidade de interações, intervalo para captura da imagem e quantidade de threads que serão usados no caso da versão paralelizada.
- Estado_inicial – fornece um estado inicial para as células vivas, representadas pelo número 1, e mortas, representadas pelo número 0, caso esse arquivo não seja disponibilizado o estado inicial é gerado de forma aleatória.

5.2. Arquivos de saída

- Tempos_parallel ou Tempos_serial – guardam os tempos gastos em computo na execução do jogo da vida.
- Imagens portable bitmap – são geradas e armazenadas conforme o intervalo indicado.

5.3. Arquivo de execução

- Makefile – usado para automatizar a execução, caso esse arquivo seja usado não é necessário o arquivo de configuração em formato de texto, na versão serial é necessário definir quantas vezes o código será executado e quais as quantidades de interações em cada simulação, na versão paralelizada também é preciso definir para quantas threads os testes serão executados.

5.4. Como executar

Usando o compilador gcc:

- Caso use as configurações por meio do arquivo de texto deve ser executado o comando " gcc grid.c image.c logic.c main.c -o gol" para a versão serial ou "gcc -fopenmp grid.c image.c logic.c main.c -o gol" para a versão paralelizada, com isso será criado um arquivo executável chamado "gol", na sequência para executar o arquivo executável usa-se o comando "./gol config.txt", esse comando já informa o arquivo de texto com as configurações.

- Caso a execuções seja realizada através do Makefile, basta usar o comando “make run”.

6. Medição de desempenho

Para a realização dos testes foram utilizadas as seguintes configurações, hardware e software:

- Cargas simuladas (número de colunas da grade): 50, 100, 500, 1000, 5000 e 10000;
- Dimensões fixas: 2000 X 2000;
- Número de threads usados: 1, 2, 4, 6, 8 e 10.
- Arquitetura: 4 núcleos físicos (com suporte a múltiplos threads)
- Processador: Intel core i5-8250U
- Velocidade: 1.60 GHz
- RAM: 8 GB
- SO: Ubuntu 24.04.2 LTS
- Compilador: GCC 13.3.0
- Processadores lógicos: 8
- Tipo de armazenamento: HD
- Capacidade de armazenamento: 410 GB

Para todos os testes realizados a grade foi dividida em quatro blocos de coluna, ou seja, granularidade quatro. A captura dos tempos de computo foram realizados de forma que fossem incluídos apenas os tempos de computo sem considerar os tempos para o salvamento de arquivos. Para isso na versão serial foi usada a função “clock()” e na versão paralela foi usado a função “omp_get_wtime()”.

A tabela 1, a seguir, mostra os tempos de execução para cada carga na versão serial do software; a tabela 2 mostra os tempos de execução em cada carga de trabalho do código paralelizado, e a tabela 3 mostra o cálculo do speedup.

Tabela 1. Tempo de execução por carga de trabalho código serial

Carga	Tempo(s)
50	40,03
100	75,22
500	334,49
1000	645,80
5000	3056,41
10000	6041,14

Tabela 2. Tempo de execução por carga de trabalho código paralelizado

Threads Carga	1	2	4	6	8	10
50	41,69	21,48	11,31	11,51	11,72	11,74
100	79,02	40,59	21,59	23,09	22,33	23,45
500	355,72	183,74	99,59	104,95	102,05	103,89
1000	689,25	358,48	189,70	193,32	198,99	198,86
5000	3277,26	1696,52	907,35	928,80	952,95	954,01
10000	6485,17	3356,23	1799,24	1863,01	1889,38	1888,94

Tabela 3. Speedup

Threads \ Carga	1	2	4	6	8	10
• 50	0,96	1,86	3,54	3,48	3,42	3,41
• 100	0,95	1,85	3,48	3,26	3,37	3,21
• 500	0,94	1,82	3,36	3,19	3,28	3,22
• 1000	0,94	1,80	3,40	3,34	3,25	3,25
• 5000	0,93	1,80	3,37	3,29	3,21	3,20
• 10000	0,93	1,80	3,36	3,24	3,20	3,20

Tabela 3. Eficiência

Threads \ Carga	1	2	4	6	8	10
• 50	96.00	92.89	88.45	57.97	42.67	34.14
• 100	95.58	92.92	87.34	54.27	42.11	32.08
• 500	94.00	91.00	83.96	53.10	40.99	32.21
• 1000	94.00	89.02	84.89	55.63	40.68	32.50
• 5000	93.00	89.73	84.35	54.83	40.08	32.03
• 10000	93.00	89.97	83.96	53.96	39.96	32.00

A seguir é apresentado o gráfico 1 mostrando o comportamento entre o tempo de execução em segundos e a carga de trabalho, do código serial. E o gráfico 2 mostra a relação entre speedup e a quantidade de threads para cada carga de trabalho.



Figure 1. Gráfico com a relação entre tempo de execução e número de iterações.

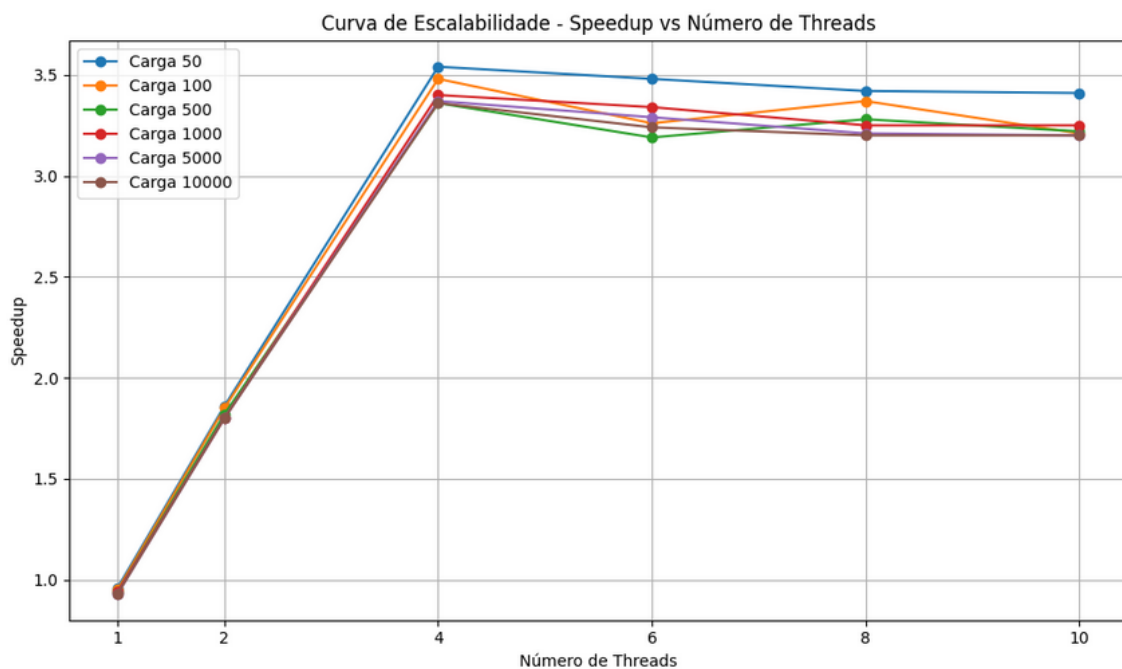


Figure 2. Gráfico com speedup para as três cargas de trabalho

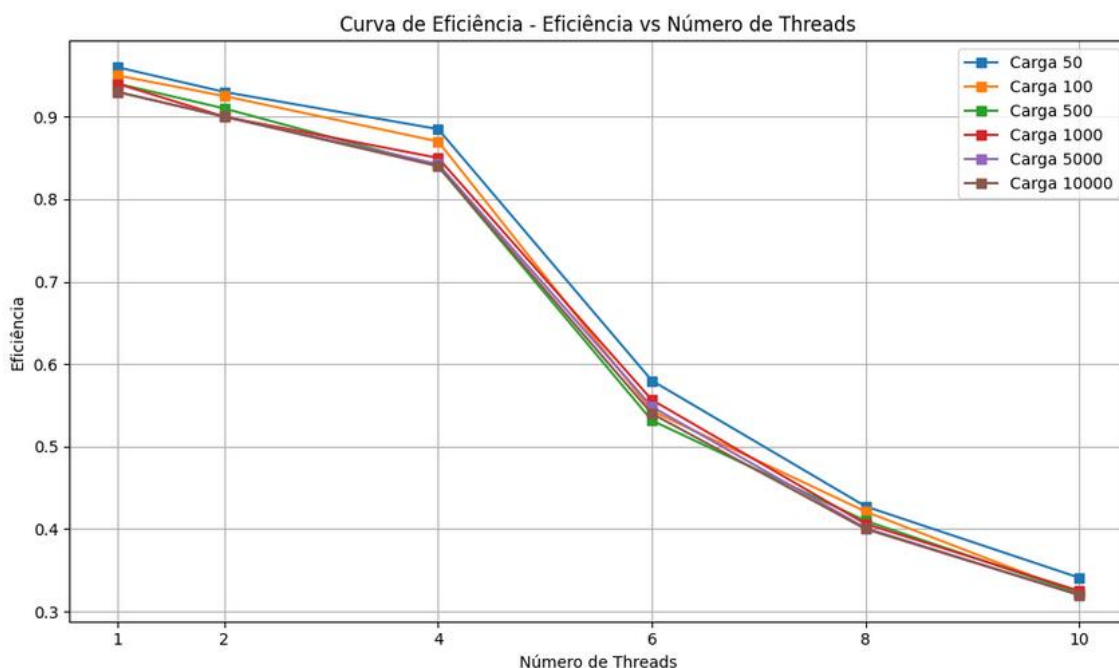


Figure 3. Gráfico com eficiência para as três cargas de trabalho

A figura 3 mostra o gráfico de eficiência no qual a eficiência atinge mais de 90% quando apenas um thread é usado, isso acontece porque os recursos computacionais estão sendo utilizados próximos do seu limite, e quando a quantidade de núcleos é aumentada os recursos computacionais são menos usados, isso é visto demonstrado no gráfico, quando 10 threads estão sendo usadas a eficiência cai para menos de 40%.

Tabela 4. Tempo de execução por granularidade

Carga	Tempo(s)-Grão 4	Tempo(s)-Grão 12
50	11.31	11.71
100	21.59	22.26
500	99.59	98.71
1000	189.70	191.75
5000	907.35	919.39
10000	1799.24	1828.09

A tabela 5 compara os tempos de execução para as cargas aplicadas, variando a quantidade de blocos de colunas em que a grade foi dividida, a tabela mostra os tempos quando se utiliza a granularidade 4 e 12. É possível observar na tabela que a granularidade 12 tem o tempo de computo é maior em comparação a granularidade 4, isso acontece por conta do aumento do tempo gasto em comunicação.

7. Análise da Escalabilidade

A escalabilidade de uma aplicação paralela descreve como seu desempenho melhora à medida que mais recursos de hardware (threads) são utilizados. A partir dos testes realizados com diferentes cargas de trabalho (número de iterações) e quantidades de threads, observamos os seguintes padrões:

- À medida que o número de threads aumenta, o tempo de execução diminui, o que resulta em um speedup crescente até certo ponto.
- Para 2 e 4 threads, o ganho de desempenho é significativo.
- A partir da quinta thread, os ganhos diminuem, indicando saturação ou sobrecarga.

Fatores que Limitam a Escalabilidade

1. Proporção de Cômputo e Comunicação
 - O cômputo (aplicação das regras do Jogo da Vida) domina o tempo de execução para cargas maiores.
 - No entanto, a comunicação implícita entre threads e a sincronização de bordas gera overhead, especialmente com mais threads.
2. Granularidade da Paralelização
 - Se o número de blocos de colunas (granularidade) for grande o desempenho começa a diminuir, o overhead de gerenciamento dos threads supera os benefícios da execução paralela.
 - Isso é demonstrado na tabela 4 que compara o tempo de execução usando 4 threads e variando a granularidade.

8. Conclusão

A partir da análise dos dados de desempenho obtidos com diferentes números de iterações e threads, podemos tirar as seguintes conclusões sobre o comportamento do algoritmo paralelizado:

A implementação com OpenMP foi capaz de reduzir o tempo de execução em comparação com a versão serial. Os maiores ganhos de performance ocorreram quando se utilizou 2 e 4 threads indicando que a maior parte do código é paralelizável.

A partir de 6 threads, o desempenho começa a estagnar ou regredir em algumas cargas, revelando os efeitos do overhead de paralelismo. Isso mostra que há um ponto ótimo de paralelismo quando se utiliza 4 threads, e com um número maior de threads os custos superam os benefícios.

Cargas maiores (mais iterações) mostram maior eficiência paralela. Isso ocorre porque, com mais trabalho a ser realizado, o custo de comunicação e sincronização se dilui em relação ao tempo de computação. Ou seja, o paralelismo é mais vantajoso com cargas intensas.

Em resumo, os resultados evidenciam que a paralelização com OpenMP é eficaz, mas seu desempenho depende da quantidade de threads e da carga de trabalho. O algoritmo alcança seu melhor aproveitamento com até 4 threads, demonstrando que existe um limite para o paralelismo eficiente, além do qual os ganhos se tornam negativos. Esse

comportamento reforça a importância de se considerar tanto o grau de paralelização quanto o perfil da aplicação ao otimizar programas para execução concorrente.

Referências

Implementing Conway's Game of Life in C, Cobb Codin. 2024